

MOVING TECH AI

Front Facing Website

Version 1.0 | Capstone Project | 2025–2026

Technology Stack: React 19 · React Router v7 · CSS Variables · Firestore (via REST API)

1. Project Overview

The front end of Moving Tech AI is a React single page application that acts as a public interface to an unbiased list of more than 150 software tools made for moving businesses. The application enables users to query, filter, and compare software profiles sourced from a Firestore based REST API.

The entire application is a read only for public users. All data comes from the companies Firestore collection as documented in Firestore schema guide. There is no form of authentication or writing operations in the front facing application.

1.1 Purpose and Scope

- Provide moving company buyers with a vendor neutral discovery tool
- Expose structured Firestore data through an accessible, filterable UI
- Display coverage scores and feature flags for each software profile
- Support continued development and hand off to the client team

1.2 Technology Stack

Technology	Role
React 19	UI component framework and rendering engine
React Router v7	Client-side routing (BrowserRouter, Routes, Route)
CSS Variables (custom)	Design token system via global.css and components.css
DM Sans / DM Mono	Typography (loaded from Google Fonts)
REST API (backend)	Firestore data consumed via fetch() calls to a FastAPI server
Create React App	Build toolchain (react-scripts 5.0.1)

2. Repository Structure

The source code lives in the `src/` directory of a standard Create React App project. The files provided for this documentation cover the complete front facing website:

File / Path	Description
<code>src/index.js</code>	Entry point - mounts React app into the DOM root
<code>src/App.js</code>	Root component - defines BrowserRouter and all Route mappings
<code>src/pages/landingPage.jsx</code>	Hero landing page with stats, preview card, and value props
<code>src/pages/directoryPage.jsx</code>	Full directory with search, filter panel, pagination, and sort
<code>src/pages/softwareDetailPage.jsx</code>	Individual software profile with feature matrix and coverage rings
<code>src/pages/comparePage.jsx</code>	Compare page (scaffolded, awaiting implementation)
<code>src/api/client.js</code>	All API fetch functions - single source of truth for HTTP calls
<code>src/hooks/useAPI.js</code>	<code>useApi</code> and <code>useDebounce</code> custom hooks
<code>src/styles/global.css</code>	Design tokens (CSS variables), resets, layout classes, page-level styles
<code>src/styles/components.css</code>	Component-scoped styles (FilterPanel, SoftwareCard, Pagination, etc.)
<code>src/styles/App.css</code>	Legacy CRA default styles (largely unused)

Note: Several component files are referenced in the pages but were not included in this documentation set. Including:

- `src/components/navbar.jsx` - Top navigation bar
- `src/components/filterPanel.jsx` - Left side filter panel (also exports `toTitleCase` and `normalizeLanguage` utilities)
- `src/components/softwareCard.jsx` - Directory listing card
- `src/components/pagination.jsx` - Page navigation controls
- `src/components/featureBadge.jsx` - Feature tag display
- `src/components/statusComponents.jsx` - `LoadingSpinner`, `ErrorMessage`, `EmptyState`

3. Routing Architecture

Routing is handled entirely by the React Router v7 using BrowserRouter. All routes are defined in App.js. The Navbar component is rendered outside the Routes block so it persists across all pages.

3.1 Route Table

Path	Component	Description
/	LandingPage	Marketing hero page - entry point for all visitors
/directory	DirectoryPage	Searchable, filterable list of all software profiles
/software/:id	SoftwareDetailPage	Full profile for one software product, identified by slug

:id parameter for the detail route corresponds to the slug of the Firestore document (smartmoving, moveboard). The navigation between pages is facilitated by the useNavigate() hook in React Router, while the useLocation() hook on the detail page accepts the prefetched state of the item using the link state of the React Router.

4. Page Components

4.1 Landing Page (landingPage.jsx)

The landing page is completely presentational in nature. It does not receive any props, and does not make any API calls either. The data displayed on the screen is hardcoded as constants in the file.

Key Sections

- Hero - two column grid with headline copy, CTA buttons, stats grid, and an animated preview card showing three sample software profiles
- Value Props - three column grid of feature highlights (Vendor neutral, Structured data, Filter by need)

Navigation Behavior

Both the CTA buttons (“Browse the Directory” and “Filter by”) will execute `navigate('/directory')` using the `useNavigate()` hook. There are no query parameters or state to be passed during navigation from the landing page.

Static Constants

Constant	Purpose
PREVIEW_ITEMS	Three sample software cards shown in the hero preview panel
STATS	Three stat tiles (150+ profiles, 12 categories, 50+ features)

4.2 Directory Page (directoryPage.jsx)

The directory page is the primary data-driven view. It manages search, filtering, pagination, and sorting state entirely within the component using React's `useState` and `useCallback` hooks.

State Variables

State	Type	Description
search	string	Raw search input value
filters	object	Active filter values (typology, best_for, install, language, status, and boolean flags)
sortBy	string	Sort key - verified_first unverified_first name_asc name_desc
page	number	Current pagination page (1-indexed)
results	object null	API response: { items: [], total: number }
options	object null	Available filter options fetched once on mount from <code>/api/software/options</code>

State	Type	Description
loading	boolean	True while an API call is in progress
error	string null	Error message if the API call failed

Data Flow

- On mount: `getFilterOptions()` is called once to populate the `FilterPanel` dropdowns
- On every change to search (debounced 350ms), filters, page, or sortBy: `fetchResults()` is triggered
- If search is non-empty, `searchSoftware()` is called; otherwise `filterSoftware()` is called with active filters
- Changing search, filters, or sortBy resets page to 1 automatically

Search Debouncing

The `useDebounce` hook from `hooks/useAPI.js` delays the search string by 350ms before it triggers an API call, preventing excessive requests while the user types.

Page Size

`PAGE_SIZE` is set to 20 results per page. This constant is defined at module scope and passed to both `searchSoftware()` and `filterSoftware()` as the `pageSize` argument.

4.3 Software Detail Page (`softwareDetailPage.jsx`)

The detail page displays the entire profile for each software product. It is navigated to with `/software/:id` and tries to receive the item object via React Router location state, passed from the directory card click, to avoid a redundant network request.

Data Loading Strategy

- If `location.state?.item` is available (navigation from directory): the item is used directly then no API call is made
- If `location.state?.item` is absent (direct URL access or refresh): `getSoftwareById(id)` is called with the url slug

Feature Matrix

The `FEATURE_SECTIONS` array defines how features are grouped and displayed. Each section contains a title, icon, a list of feature keys, and human readable labels:

Section	Icon	Feature Keys
Sales	◆	lead_mgmt, surveying, quoting
Move Management	⬡	job_booking, move_mgmt
Operations	⦿	dispatch, shipments, crew_app, storage
ERP / Back Office	■	ar, ap, hr

Coverage Rings

CoverageRing is an SVG based animated donut chart drawn inline. It takes a score between 0 to 100 and a label, and paints itself green for a score $\geq 75\%$, amber $\geq 40\%$ score, and red $< 40\%$. The rings are displayed for overall coverage and one for each feature section.

Feature Value Display

The FeatureValue component translates backend values into UI states. For 'Y', it shows a Yes badge in green, any other non null value displays a No badge in red, and null or undefined displays a neutral dash (-).

Navigation

The Back to Directory button utilizes `navigate(-1)` to go back to the previous history entry and retains the directory scroll position and filter state.

5. API Client (client.js)

The entire HTTP requests to the backend are encapsulated within the `src/api/client.js` file. There are four asynchronous functions that have been exported from the file. Each of these functions makes a call to the internal `request()` function, which builds the complete URL based on the value of the environment variable `REACT_APP_API_URL` (default: `http://localhost:8000`).

5.1 Environment Configuration

The base URL is obtained from the `REACT_APP_API_URL` environment variable. The value of the `REACT_APP_API_URL` variable needs to be set to the URL of the deployed backend service when deploying the application on Render.

```
REACT_APP_API_URL=https://your-backend-service.onrender.com
```

5.2 API Functions

Function	Endpoint	Description
<code>searchSoftware(q, page, pageSize, sortBy)</code>	GET <code>/api/software/search</code>	Full-text search by name, description, or keywords. Returns paginated results.
<code>filterSoftware(filters, page, pageSize, sortBy)</code>	GET <code>/api/software/filter</code>	Faceted filter by typology, market, install type, and feature flags. Returns paginated results.
<code>getFilterOptions()</code>	GET <code>/api/software/options</code>	Returns available values for all filter dropdowns. Called once on <code>DirectoryPage</code> mount.
<code>getSoftwareById(id)</code>	GET <code>/api/software/:id</code>	Fetches a single software profile by its Firestore slug. Used on the detail page for direct URL access.

5.3 Query Parameters

`searchSoftware` and `filterSoftware` both accept the following common parameters:

Parameter	Default	Description
<code>page</code>	1	Page number (1-indexed)
<code>pageSize</code>	20	Number of results per page
<code>sort_by</code>	<code>verified_first</code>	<code>verified_first</code> <code>unverified_first</code> <code>name_asc</code> <code>name_desc</code>

`filterSoftware` additionally appends one query parameter per active filter key. Array-valued filters (typology, best_for, install, language, status) are appended multiple times with the same key name using `URLSearchParams.append()`.

6. Custom Hooks (useAPI.js)

6.1 useApi

Generic hook for fetching data from any asynchronous function and returning { data, loading, error, refetch }. It makes use of mountedRef to prevent state updates after component unmounting to avoid memory leaks during page navigations.

DirectoryPage doesn't make use of useApi; it maintains its own state using useState and useCallback for more granular control over when the API should be re-fetched. The useApi hook can be used by other components if needed.

6.2 useDebounce

Delays a value update by a configurable number of milliseconds (default 350ms). Used in DirectoryPage to prevent an API call on every keystroke:

```
const debouncedSearch = useDebounce(search, 350);
```

The debounced value is what triggers the fetchResults useCallback, not the raw search string.

7. Styling System

The application uses a pure CSS design token system - no CSS-in-JS library, no Tailwind. Styles are split across two files:

7.1 global.css

Includes all that is relevant at the page level or above:

- Custom CSS properties (:root variables) — single source of truth for all colors, spacings, shadows, and fonts
- Reset (box-sizing, margin, padding)
- Default body settings (font, background color, line height, 68px padding at the top to counteract the fixed navbar)
- Button styles (.btn, .btn--primary, .btn--ghost)
- Page layouts for the Landing, Directory, and Detail pages
- Sorting component styles and result header styles

7.2 Design Token Reference

Token	Value	Used For
--navy	#0f1c2e	Primary dark background (headers, topbars)
--accent	#2563eb	Primary blue - CTAs, links, feature icons
--green	#10b981	Positive states, verified badges, coverage high
--amber	#f59e0b	Warning states, unverified badges, coverage mid
--red	#ef4444	Negative feature flags, coverage low
--bg	#f5f6f8	Page background (light sections)
--surface	#ffffff	Card and panel backgrounds
--border	#e2e6ec	Default border color
--text-primary	#0d1117	Main body text
--text-secondary	#4a5568	Supporting text
--text-muted	#8592a3	Labels, captions, de-emphasized text
--font-sans	DM Sans	All UI text
--font-mono	DM Mono	Code-style labels

7.3 components.css

Contains styles scoped to reusable UI components including the FilterPanel sidebar, SoftwareCard grid items, Pagination controls, feature badges, and status indicators. Component

classes follow a BEM-like naming convention consistent with the page-level classes in `global.css`.

7.4 Responsive Breakpoints

Breakpoint	Behavior
max-width: 900px	Directory body switches from sidebar+results to single column. Hero preview card hidden.
max-width: 700px	Value props grid collapses to single column
max-width: 600px	Software grid collapses to single column
max-width: 860px	Overview cards grid on detail page switches to 2-column
max-width: 500px	Overview cards grid collapses to single column

8. Data Model: Frontend ↔ Firestore Mapping

The front end accesses data through the backend REST API that converts Firestore document data into the JSON format. Below is the correspondence between the front-end field names used in the components and their Firestore paths as per the accompanying schema.

Frontend Field	Firestore Path	Used In
item.id	Document ID (slug)	React key, URL param, getSoftwareById()
item.name	name	Card title, detail header
item.description	description	Card summary, detail header
item.website	website	Detail page external link
item.typology	software_typology	Card badge, filter, detail typology badge
item.best_for	target_market (joined string)	Overview card - Total Market
item.install	deployment_types (joined string)	Overview card - Deployment
item.language	languages (joined string)	Overview card - Languages
item.keywords	keywords (comma-separated string)	Overview card, search indexing
item.status	status.review_status	Status badge (verified / unverified)
item.features.lead_mgmt	features.sales.lead_management	Feature matrix - Sales section
item.features.surveying	features.sales.surveying	Feature matrix - Sales section
item.features.quoting	features.sales.quoting	Feature matrix - Sales section
item.features.job_booking	features.move_management.job_booking	Feature matrix - Move Mgmt section
item.features.move_mgmt	features.move_management.move_mgmt	Feature matrix - Move Mgmt section
item.features.dispatch	features.operations.dispatch	Feature matrix - Operations section
item.features.shipments	features.operations.shipment	Feature matrix - Operations section
item.features.crew_app	features.operations.crew_app	Feature matrix - Operations section
item.features.storage	features.operations.storage	Feature matrix - Operations section
item.features.ar	features.erp.accounts_receivable	Feature matrix - ERP section
item.features.ap	features.erp.accounts_payable	Feature matrix - ERP section
item.features.hr	features.erp.hr	Feature matrix - ERP section

Key point: The backend API converts all the array fields, target market, deployment types, languages, and keywords, into comma-delimited strings before sending the data to the frontend. The front end then converts the string back to an array where needed, such as displaying the keywords on softwareDetailPage.jsx.

9. Filter and Search System

9.1 Filter State Shape

The INITIAL_FILTERS constant in directoryPage.jsx defines the default filter state object:

```
{ typology: [], best_for: [], install: [], language: [], status: [],  
  lead_mgmt: null, dispatch: null, crew_app: null, storage: null, surveying: null  
}
```

Array fields accept multiple selected values (multi-select filter). Boolean fields (lead_mgmt, dispatch, crew_app, storage, surveying) accept null (no filter), true (must have), or false (must not have).

9.2 Search vs. Filter Mode

The directory can be in one of two possible, mutually exclusive states:

- Search state: invoked if debouncedSearch is not empty. Uses searchSoftware(). Does not use activeFilters.
- Filter state: invoked if debouncedSearch is empty. Uses filterSoftware(). Filters that are not active are filtered out before constructing the query.

Clearing the search input automatically returns to filter mode on the next debounce cycle.

9.3 Sort Options

Sort Value	Behavior
verified_first	Profiles with status = 'verified' appear before 'unverified' (default)
unverified_first	Unverified profiles appear first - useful for editorial review
name_asc	Alphabetical A to Z by software name
name_desc	Alphabetical Z to A by software name

10. Coverage Score Calculation

Coverage score is calculated in the frontend based on the feature flags obtained via the API call. The backend also maintains precomputed values of `coverage_score` in Firestore (refer to schema documentation); however, the detailed view calculates the coverage score again using feature information dynamically on the client side.

10.1 Calculation Logic

`ComputeOverallCoverage(features)`

- Loops through all the keys in all four `FEATURE_SECTIONS`.
- Counts the number of non-null values equal to 'Y'.
- Returns the result as a percentage (nearest integer).

`ComputeSectionScore(section, features)`

- Same principle as above for just one section

Nulls do not affect the calculation in either the numerator or denominator.

10.2 Color Thresholds

Score Range	Color	Token
75% – 100%	Green	#10b981 (--green)
40% – 74%	Amber	#f59e0b (--amber)
0% – 39%	Red	#ef4444 (--red)

11. Local Development Setup

11.1 Prerequisites

- Node.js 18 or later
- npm 9 or later
- The backend API running locally (see backend documentation) or accessible via a deployed URL

11.2 Installation

```
# Clone the repository
git clone <repository-url>
cd <frontend-directory>

# Install dependencies
npm install
```

11.3 Environment Variables

Create a .env file in the project root (same level as package.json):

```
REACT_APP_API_URL=http://localhost:8000
```

For connecting to the deployed Render backend during local development:

```
REACT_APP_API_URL=https://your-backend.onrender.com
```

11.4 Running the Dev Server

```
npm start
```

The app will open at <http://localhost:3000>. The dev server proxies requests and hot-reloads on file save.

11.5 Building for Production

```
npm run build
```

Outputs a production optimized build to the build/ directory. This is the artifact deployed to Render (or any static hosting service).

12. Deployment on Render

Deployment for the website frontend has been done using Static Sites on Render. It refers to the GitHub repository that is used as the source code for the backend. Deployment is automated whenever there are changes pushed to the designated branch.

12.1 Render Static Site Settings

Setting	Value
Service Type	Static Site
Build Command	npm install && npm run build
Publish Directory	build
Branch	main (or as configured)
REACT_APP_API_URL	https://<backend-service>.onrender.com (set as environment variable)

12.2 SPA Redirect Rule

Because the app uses client-side routing (BrowserRouter), direct URL access to paths like /directory or /software/smartmoving would result in a 404 from the static server. A Render redirect rule must be configured:

Source	Destination	Status	Purpose
/*	/index.html	200 (rewrite)	Serves React app for all routes, allowing client-side routing to handle the path

This is configured in the Render dashboard under Redirects/Rewrites for the static site service.

12.3 Updating the Frontend

- Push changes to the configured GitHub branch
- Render automatically detects the push and triggers a new build
- Build logs are available in the Render dashboard Events tab
- If the build succeeds, the new version is live within 1–3 minutes
- If the build fails, the previous deploy remains live - no downtime

13. Compare Page (Future Implementation)

The `comparePage.jsx` is just a blank scaffold now. While it's listed as a route within `App.js`, it has no code implemented at all. According to the requirements of the project specification, this component should be a feature where you can compare two or more software profiles side by side.

13.1 Suggested Implementation Approach

- Add a button to each `SoftwareCard` in the grid called `Compare`
- Define a `compareList` array of size up to 2 to 4 using React state (or React Context) in the `App` component
- The compare page will fetch the list and execute the function `getSoftwareById()` on each list item
- Show a table where rows represent features and columns software products, utilizing the existing `FeatureValue` component for each element
- Use the same `FEATURE_SECTIONS` array defined in the `softwareDetailPage.jsx` as the rows definition

14. Known Limitations and Future Work

14.1 Current Limitations

- Search doesn't account for filters – it disregards any currently selected filters and forces the user to remove the filter or undo the search. A feature to implement could be to combine both functionalities within one query.
- The comparePage.jsx file serves as a blank template and doesn't work.
- The keywords attribute in Firestore is represented as an array (String[]) but on the front-end side, we receive a string. It is a bug that needs to be fixed.
- ERP module keys (ar, ap, hr) in FEATURE_SECTIONS are different from those used in the Firestore structure (accounts_receivable, accounts_payable, and hr). It is another issue with backend API.
- There is no functionality implemented for service workers/offline capability.

14.2 Recommended Future Improvements

- Unified search + filters: integrate both the search criteria and filters into one call on the backend
- Comparison page: add implementation for the comparison page design discussed in section 13
- Bookmarking comparisons: ability to bookmark comparison pages
- Reviews system: frontend integration with reviews sub-collection that we plan on implementing in our firestore schema
- Vendors' submit form: vendor form where vendors can submit their information independently of other forms
- Analytics: track user traffic and terms used in the searches

15. Quick Reference

Key File Locations

Need to change...	Edit this file
Backend API URL	.env → REACT_APP_API_URL
Routes / navigation	src/App.js
Landing page copy or stats	src/pages/landingPage.jsx (PREVIEW_ITEMS, STATS constants)
Filter options or sort options	src/pages/directoryPage.jsx (INITIAL_FILTERS, SORT_OPTIONS)
Feature sections or labels	src/pages/softwareDetailPage.jsx (FEATURE_SECTIONS)
API endpoints	src/api/client.js
Colors / design tokens	src/styles/global.css (:root variables)
Component styles	src/styles/components.css
Results per page	src/pages/directoryPage.jsx (PAGE_SIZE constant)
Coverage score thresholds	src/pages/softwareDetailPage.jsx (CoverageRing, SectionCoverageBar)

- End of Document -